

Bisou App

Detaillierte Funktionsspezifikation & Systemdokumentation

Entwickler & Architekt: Benedikt S.

Dokumentation erstellt durch: Antigravity AI Partner

Datum: 20. Juni 2026

Version: 6.0 (Optimiert auf Hauptfunktionen & lückenlose Detailtiefe)

Stack: React Frontend & Supabase (Postgres Database, Edge Functions) Backend

1. Das Fragensystem & die Gemini-KI-Generierung

Die tägliche Interaktion basiert auf vier verschiedenen Fragentypen, die Paare auf emotionale und spielerische Weise einander näherbringen. Die Bereitstellung der Fragen erfolgt vollautomatisch über eine Supabase Edge Function (generate-questions), die über einen täglichen Cron-Job (mittels pg_cron auf der Datenbank) um 12:00 Uhr mittags getriggert wird.

1.1 JSON-Struktur der täglichen Fragen

In der Tabelle daily_questions werden die Fragen im Feld questions als strukturiertes JSONB-Dokument gespeichert. Dies ermöglicht eine flexible Auswertung und Übergabe an das Frontend. Ein valides JSON-Dokument besitzt folgendes Schema:

```
{
  "tot": {
    "q": "Was ist euch an einem Sonntag wichtiger?",
    "o": ["Gemütliches Ausschlafen & Frühstück im Bett", "Früh aufstehen & zusammen etwas erleben"]
  },
  "ranking": {
    "q": "Sortiert diese Beziehungsfaktoren nach eurer persönlichen Priorität:",
    "o": ["Gemeinsame Hobbys", "Offene Kommunikation", "Körperliche Nähe", "Gegenseitiger Freiraum"]
  },
  "text": {
    "q": "Was war ein kleiner Moment in dieser Woche, für den du deinem Partner dankbar bist?"
  },
  "wwe": {
    "q": "Wer von euch beiden verliert bei einem Brettspiel eher die Geduld?"
  }
}
```

1.2 Der Prompt-Algorithmus der Gemini API

Falls für ein angefordertes Datum (day_key) noch kein Fragen-Datensatz existiert, formuliert die Edge Function einen detaillierten System-Prompt an das Modell gemini-1.5-flash. Der Prompt instruiert die KI wie folgt:

- Generiere genau 4 Fragen mit den Typen tot (Dies oder Das mit 2 Optionen), ranking (mit exakt 4 Optionen), text (offene Freitextfrage) und wwe (Wer würde eher).
- Die Fragen müssen sich auf die Themen Partnerschaft, Emotionen, Alltag, Träume und gemeinsame Zukunft beziehen. Sie müssen abwechslungsreich sein und dürfen sich niemals wiederholen. Plattitüden oder rein materielle Abfragen (z.B. Automarken) sind verboten.
- Erzwing die Ausgabe als reines, valides JSON-Dokument ohne Markdown-Formatting (keine Backticks wie ``json), damit der Deno-Server die Antwort ohne reguläre Ausdrücke parsen und direkt in die Datenbank schreiben kann.

1.3 Robuste Fallbacks & Fehlerbehandlung

Um einen Systemausfall bei API-Drosselungen oder Netzwerkstörungen der Google API zu verhindern, implementiert die Funktion ein mehrstufiges Sicherheitsnetz:

1. **Lokale Fallback-Datenbank:** Im Code der Edge Function ist eine statische Konstante FALLBACK_QUESTIONS mit vordefinierten, hochwertigen Beispielfragen hinterlegt. Schlägt der Gemini-Aufruf fehl, greift das System auf diese zurück, sodass der Benutzer keine Fehlermeldung sieht.
2. **Wiederholungs-Warteschlange (Retry-Queue):** Schlägt die Generierung fehl, schreibt die Funktion einen Datensatz mit dem betroffenen Datum in die Tabelle failed_generations. Ein Datenbank-Cronjob (retry_failed_generations) prüft diese Tabelle alle 10 Minuten und versucht, die fehlenden Fragen asynchron im Hintergrund erneut per KI zu generieren. Ist dies erfolgreich, wird der Queue-Eintrag gelöscht.

2. Der Beantwortungsprozess & die Datenserialisierung

Der Beantwortungsprozess im Client wird über einen Wizard gesteuert. Die App stellt sicher, dass Teilantworten lokal geschützt sind und dass die Antworten beider Partner sicher miteinander verglichen werden können.

2.1 Clientseitige Zustandssicherung (LocalStorage-Cache)

Während der Benutzer die Fragen beantwortet, speichert die Komponente Questions.tsx nach jedem Einzelschritt den aktuellen Zustand im LocalStorage (Schlüssel: quiz_progress_{dayKey}). Das Objekt speichert den aktuellen Index des Schritts (0 bis 3) und ein Array mit den bisherigen Antworten. Dadurch wird verhindert, dass der Benutzer bei einem Verbindungsabbruch, einem versehentlichen Schließen der App oder dem Neuladen der Seite seinen Fortschritt verliert. Sobald der letzte Schritt vollendet und die Antworten erfolgreich an die Supabase-Datenbank übermittelt wurden, wird dieser LocalStorage-Eintrag bereinigt.

2.2 Das serielle Antwort-String-Format & Signaturen

Die Antworten eines Benutzers für einen Tag werden nicht als separate Spalten oder Zeilen abgelegt, sondern als ein einziger, strukturiert serialisierter Text-String in der Spalte choice der Tabelle answers gespeichert. Das Format ist wie folgt aufgebaut: Antwort1 | Antwort2 | Antwort3 | Antwort4 [Frage1][Frage2][Frage3][Frage4]

Beispiel für einen echten Datenbank-Eintrag:

Sofa & Netflix | Offene Kommunikation > Körperliche Nähe > Hobbys | Zusammen kochen | Partner [tot:Was macht ihr sonntags?][ranking:Sortiert nach Prio...][text:Lieblingsgericht?][wwe:Wer ist ungeduldiger?]

Der Zweck der Signatur-Klammern (Spionschutz & Integrität):

Die am Ende angehängten Signatur-Klammern [...] enthalten den exakten Fragetext zum Zeitpunkt der Beantwortung. Da die Fragen theoretisch im Laufe des Tages regeneriert werden könnten (z. B. durch manuelle Admin-Eingriffe oder System-Resets), dient die Signatur als Validierung. Die Auswertungs-Funktion calculate-stats vergleicht die Signaturen beider Partner. Stimmen die Signaturen nicht überein, erkennt das System, dass die Partner auf unterschiedliche Fragen geantwortet haben. Dies verhindert Berechnungsfehler und falsche Auswertungen bei asynchronen Datumswechseln.

2.3 Ablauf beim Absenden der Daten

Sobald der Benutzer den letzten Schritt abschließt, führt die App eine Datenbanktransaktion aus:

1. Löscht eventuell bereits vorhandene temporäre Antworten des Benutzers für diesen Tag (falls korrigiert wird).
2. Fügt den neu generierten Antwort-String in public.answers ein.
3. Triggert über die Edge-Function eine Push-Benachrichtigung an den Partner, dass die Antworten vorliegen.
4. Startet die lokale clientseitige Verschlüsselungs-Animation und ruft onComplete() auf, um das Dashboard zu aktualisieren.

3. Der Kompatibilitäts- & Matching-Algorithmus (calculate-stats)

Die Berechnung der Übereinstimmungswerte erfolgt in der Deno Edge Function calculate-stats. Sie analysiert die letzten 30 Tage der Beziehung und wendet für jeden Fragentyp spezifische mathematische Verfahren an.

3.1 Die vier Abgleich-Methoden im Detail

- **Dies oder Das (tot) - Binär-Matching:** Ein einfacher String-Vergleich. Stimmen die ausgewählten Optionen überein, beträgt der Match-Wert für diesen Tag 100%, andernfalls 0%.
- **Ranking-Frage - Spearman-Distanz mit Wurzel-Dämpfung:** Die Partner sortieren 4 Elemente. Die Abweichung wird über die Summe der quadrierten Differenzen der Ränge (1 bis 4) berechnet. Der maximal mögliche quadratische Abstand bei 4 Elementen beträgt $d^2_{\text{max}} = 20$ (wenn die Listen komplett entgegengesetzt sortiert sind). Um eine Überstrafung kleiner Abweichungen zu verhindern und Paare psychologisch zu ermutigen, wird die Ähnlichkeit über eine Wurfelfunktion geglättet: $\text{Score} = \sqrt{1 - (\text{Summe}(\text{Rang_A} - \text{Rang_B})^2 / 20)} * 100$
- **Freitext-Frage - Semantische Kosinus-Ähnlichkeit:** Die Texte werden durch das in der Edge Function ausgeführte Embedding-Modell gte-small in 384-dimensionale Vektor-Arrays konvertiert. Die mathematische Ähnlichkeit ist das Skalarprodukt der normierten Vektoren:
$$\text{CosineSimilarity} = (A \cdot B) / (||A|| * ||B||)$$
Ein identischer Text liefert 1.0 (100%), völlig unterschiedliche Texte liegen meist bei 0.3. Das System mappt den Bereich [0.3; 0.9] linear auf [0%; 100%]. Schlägt die Generierung fehl (z.B. Timeout), berechnet das System als Fallback den Jaccard-Koeffizienten (Schnittmenge der Wörter geteilt durch die Vereinigungsmenge).
- **Wer würde eher (wwe) - Komplementär-Abgleich:** Da sich Partner bei der Frage 'Wer verliert eher die Geduld?' einig sind, wenn einer 'Ich' und der andere 'Partner' wählt, liegt ein Match vor, wenn die Antwort-Strings *ungleich* sind: `choice_A !== choice_B` (Match: 100%, sonst 0%).

3.2 Dynamische Gewichtung & Berechnung des Bisou-Scores

Der Gesamt-Bisou-Score liegt auf einer Skala von 0.0 bis 10.0. Jede der vier Fragenkategorien hat das gleiche Gewicht von 25% (Faktor 0.25). Um verfälschte Ergebnisse bei unvollständigen Daten (z. B. wenn an einem Tag keine Freitext-Frage gestellt wurde oder die Partner die wwe-Frage übersprungen haben) zu vermeiden, wendet die Funktion eine **dynamische Gewichtungs-Normalisierung** an:

- Es werden nur Kategorien in die Berechnung einbezogen, bei denen für die betrachteten Tage tatsächlich Antworten beider Partner vorliegen.
- Die Summe der Übereinstimmungsprozente der aktiven Kategorien wird durch die Anzahl der tatsächlich aktiven Kategorien geteilt.
- Der resultierende Prozentwert wird durch 10 geteilt und mathematisch auf eine Dezimalstelle gerundet (z.B. 84.3% Übereinstimmung = Score 8.4).

3.3 Trendberechnung und Trendpfeile

Das Dashboard zeigt neben dem Bisou-Score einen Trendpfeil (Up/Down) an. Um diesen zu ermitteln, berechnet die Edge Function zwei Werte parallel: Den aktuellen Bisou-Score der letzten 30 Tage (inklusive heute) und den vorherigen Bisou-Score (indem der allerneueste gemeinsame Antworttag aus der Berechnung ausgeschlossen wird). Ist der aktuelle Score höher als der vorherige, wird ein steigender Trend signalisiert, andernfalls ein fallender. Dies motiviert Paare, durch kontinuierlich übereinstimmende Antworten ihren Score aktiv zu steigern.

4. Frontend-Architektur & Design-Spezifikation

Das visuelle Design der Bisou App zeichnet sich durch weiche Verläufe, harmonische Farbtöne und intuitive Bedienung aus.

4.1 HSL-Theme Farbvariablen

Das CSS-Theme steuert alle Farben über native Variablen, um eine nahtlose Umschaltung zwischen Hell- und Dunkelmodus zu ermöglichen. Die Primärfarbe (Rose/Rot) vermittelt Wärme, während die Sekundärfarbe (Soft Lila) für Vertrauen steht.

CSS Variable	Light Theme	Dark Theme	Semantischer Nutzen
--primary	#FF8A8A (Pastell-Rot)	#FF9B9B (Hell-Rot)	Streaks, Buttons, Akzente
--secondary	#A29BFE (Soft-Lila)	#B4AFFE (Hell-Lila)	Buttons, Meilensteine
--bg	#F8F7FF (Pastell-Weiß)	#0C0A15 (Tief-Schwarz)	Hintergrundfarbe
--text-main	#1F1939 (Dunkel-Lila)	#F5F3FF (Off-White)	Überschriften und Fließtext
--card-border	#E2DFFF (Hell-Grau-Lila)	#231E3D (Dunkel-Violett)	Ränder von Karten & Feldern

4.2 Typografie

- **Fraunces:** Eine ausdrucksstarke Serifenschrift. Sie wird im Header für das 'Bisou'-Logo und ausgewählte emotionale Überschriften verwendet, um Intimität und Individualität zu vermitteln.
- **Plus Jakarta Sans:** Eine hochmoderne, geometrische Sans-Serif-Schrift. Sie dient als Haupt-Schriftart der Benutzeroberfläche und sorgt dank großzügiger Zeichenbreiten für hervorragende Lesbarkeit auf Smartphone-Bildschirmen.

4.3 Micro-Animations & Interaktionseffekte

- **Wobble & Flicker (`.animate-flicker`):** Animiert die Flammen-Icons bei aktiven Antwortserien mit zufallsnahen Rotationsänderungen (± 3 Grad), um das Flackern einer echten Kerze nachzuahmen.
- **Eingangs-Transitionen (`.animate-fade-in-up`):** Karten und Formularelemente gleiten beim Laden um 24 Pixel von unten nach oben und blenden gleichzeitig ein. Eine gestaffelte CSS-Verzögerung (Stagger-Effekt) von 80ms pro Element lässt die Seiten organisch und flüssig aufgebaut erscheinen.

5. Datenbank-Struktur & Schema-Referenz

Die PostgreSQL-Datenbank strukturiert alle Relationen, Integritätsbedingungen (Foreign Keys) und Trigger.

5.1 Profile-Tabelle (public.profiles)

```
CREATE TABLE public.profiles (  
  id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  display_name text,  
  partner_id uuid REFERENCES public.profiles(id) ON DELETE SET NULL,  
  partner_code text UNIQUE,  
  avatar_url text,  
  intro_completed boolean DEFAULT false NOT NULL,  
  partner_since timestampz,  
  last_answer_reset_at timestampz,  
  nudge_cooldown timestampz,  
  nudge_count integer DEFAULT 0 NOT NULL,  
  last_nudge_at timestampz,  
  total_answers integer DEFAULT 0 NOT NULL,  
  total_matches integer DEFAULT 0 NOT NULL,  
  nudges_sent integer DEFAULT 0 NOT NULL,  
  morning_answers_count integer DEFAULT 0 NOT NULL,  
  night_answers_count integer DEFAULT 0 NOT NULL,  
  lunch_answers_count integer DEFAULT 0 NOT NULL,  
  last_minute_answers_count integer DEFAULT 0 NOT NULL,  
  long_answers_count integer DEFAULT 0 NOT NULL,  
  both_long_answers_count integer DEFAULT 0 NOT NULL,  
  journal_views integer DEFAULT 0 NOT NULL,  
  streaks_rebuilt integer DEFAULT 0 NOT NULL,  
  avatar_change_count integer DEFAULT 0 NOT NULL,  
  time_sync_5min_count integer DEFAULT 0 NOT NULL,  
  time_sync_1min_count integer DEFAULT 0 NOT NULL,  
  answered_valentines boolean DEFAULT false NOT NULL,  
  answered_new_years boolean DEFAULT false NOT NULL,  
  perfect_rankings_count integer DEFAULT 0 NOT NULL,  
  perfect_match_days_count integer DEFAULT 0 NOT NULL  
);
```

5.2 Antworten-Tabelle (public.answers)

```
CREATE TABLE public.answers (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,  
  day_key date NOT NULL,  
  choice text NOT NULL,  
  created_at timestampz DEFAULT timezone('utc'::text, now()) NOT NULL,  
  CONSTRAINT answers_user_id_day_key_key UNIQUE (user_id, day_key)  
);
```

5.3 Streaks-Tabelle (public.streaks)

```
CREATE TABLE public.streaks (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,  
  partner_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE,  
  current_streak integer DEFAULT 0 NOT NULL,  
  longest_streak integer DEFAULT 0 NOT NULL,  
  last_answer_date date,  
  streak_history jsonb DEFAULT '[]'::jsonb NOT NULL,  
  CONSTRAINT streaks_user_id_partner_id_key UNIQUE (user_id, partner_id)  
);
```

5.4 Zusätzliche Tabellen-Spezifikationen

```
-- Ankündigungen (Server-Driven Popups)
CREATE TABLE public.announcements (
  id bigint PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  version_code integer UNIQUE NOT NULL,
  title text NOT NULL,
  content text NOT NULL,
  type text DEFAULT 'info'::text NOT NULL,
  emoji text DEFAULT '■'::text NOT NULL,
  button_label text DEFAULT 'Los geht's'::text NOT NULL,
  action_route text,
  created_at timestamptz DEFAULT now() NOT NULL
);

-- Ankündigung-Views (Gelesen-Status)
CREATE TABLE public.announcement_views (
  id bigint PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  user_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  announcement_id bigint REFERENCES public.announcements(id) ON DELETE CASCADE NOT NULL,
  seen_at timestamptz DEFAULT now() NOT NULL,
  CONSTRAINT announcement_views_user_id_announcement_id_key UNIQUE (user_id, announcement_id)
);

-- Push-Abonnements
CREATE TABLE public.push_subscriptions (
  user_id uuid PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,
  subscription jsonb NOT NULL,
  created_at timestamptz DEFAULT now() NOT NULL
);

-- Erfolge / Meilensteine Definitionen
CREATE TABLE public.milestones (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  description text NOT NULL,
  icon text NOT NULL,
  trigger_type text NOT NULL,
  trigger_value integer NOT NULL,
  created_at timestamptz DEFAULT now() NOT NULL
);

-- Freigeschaltete Erfolge
CREATE TABLE public.unlocked_milestones (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  milestone_id uuid REFERENCES public.milestones(id) ON DELETE CASCADE NOT NULL,
  unlocked_at timestamptz DEFAULT now() NOT NULL,
  is_seen boolean DEFAULT false NOT NULL,
  CONSTRAINT unlocked_milestones_user_id_milestone_id_key UNIQUE (user_id, milestone_id)
);
```

6. RLS (Row Level Security) & Zugriffskontrolle

Die Sicherheit der Benutzerdaten ist über restriktive Postgres-RLS-Regeln geregelt. Jeder Nutzer kann ausschließlich seine eigenen Einträge modifizieren, während Lesezugriffe für Partner über SQL-Unterabfragen freigegeben werden.

Tabelle	Operation	SQL Policy Bedingung (USING / WITH CHECK)
profiles	SELECT INSERT/UPDATE DELETE	TO authenticated USING (true) USING (auth.uid() = id) USING (auth.uid() = id)
answers	SELECT INSERT/DELETE	auth.uid() = user_id OR user_id IN (SELECT partner_id FROM public.profiles WHERE id = auth.uid()) USING (auth.uid() = user_id)
streaks	SELECT	auth.uid() = user_id OR user_id IN (SELECT partner_id FROM public.profiles WHERE id = auth.uid())
announcements	SELECT	TO public USING (true)
announcement_views	SELECT INSERT	USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id)
milestones	SELECT	TO authenticated USING (true)
unlocked_milestones	SELECT INSERT	TO authenticated USING (true) TO service_role WITH CHECK (true)

7. PL/pgSQL Funktionen, Trigger & RPCs

Komplexe Geschäftslogik und relationale Updates werden direkt im PostgreSQL-Server ausgeführt. Dies vermeidet Race-Conditions und minimiert Roundtrips vom Client.

7.1 Partnerkopplung: `public.link_partners()`

Führt eine atomare, bidirektionale Verknüpfung zweier Profile durch. Die Funktion nimmt den ``partner_code_to_link`` entgegen, sucht die zugehörige UUID und setzt gegenseitig ``partner_id`` sowie ``partner_since`` auf den aktuellen Zeitstempel.

7.2 Kontolöschung: `public.delete_user_account()`

Löscht das Konto des angemeldeten Benutzers sicher aus ``auth.users``. Um eine Verletzung von Fremdschlüsselbedingungen (FK Violation) zu vermeiden, hebt die Funktion zuerst die Koppelung beim Partner auf (setzt dort ``partner_id`` auf NULL) und löscht anschließend den User-Datensatz. Kaskadierende Löschungen leeren automatisch verknüpfte Tabellen (Answers, Streaks).

7.3 Streak-Verwaltung: `public.update_streak()`

Ein AFTER INSERT-Trigger auf ``public.answers``. Wird eine Antwort gespeichert, ermittelt der Trigger das aktuelle Datum und prüft den Tag der letzten Antwort. War die letzte Antwort gestern, wird der Streak inkrementiert und das Datum an die ``streak_history`` angehängt. Liegt die letzte Antwort länger zurück, wird die Serie auf 1 zurückgesetzt. Das System stellt sicher, dass Korrekturen am selben Tag den Streak nicht brechen.

7.4 Antwort-Reset: `public.reset_today_answers(day_key_param text)`

Erlaubt es Nutzern, ihre heutigen Antworten zu löschen, um sie neu zu beantworten. Die Funktion implementiert einen harten 7-Tage-Cooldown. Sie prüft ``profiles.last_answer_reset_at``. Liegt der Wert weniger als 7 Tage in der Vergangenheit, wird die Ausführung mit einer Fehlermeldung (RAISE EXCEPTION) abgebrochen, die dem Nutzer die verbleibenden Tage/Stunden anzeigt.

7.5 Cronjobs (Datenbereinigung & Retry-Queue)

- **`public.cleanup_old_answers()`**: Löscht täglich um 3:00 Uhr alle Antworten, die älter als 90 Tage sind. Dies schont Speicherressourcen und wahrt die Datensparsamkeit.
- **`public.retry_failed_generations()`**: Wird alle 10 Minuten ausgeführt. Sucht in ``failed_generations`` nach Fehlversuchen und sendet mittels der PostgreSQL-Erweiterung `pg_net` einen neuen asynchronen HTTP-POST-Request an die Edge Function zur Fragengenerierung.

7.6 Meilenstein-Freischaltung: `check_and_unlock_milestones()`

Ein vollautomatisches Daten-Auswertungssystem auf Datenbankebene. Um die historische Unvollständigkeit der Tabelle `answers` zu umgehen, erfasst das System alle Fortschritte über persistente Zähler-Felder in der Tabelle `profiles`. Diese Felder werden automatisch über verschiedene Trigger gewartet:

- **Antworten & Habits**: `total_answers`, `morning_answers_count`, `night_answers_count`, `lunch_answers_count` und `last_minute_answers_count` werden bei jedem Antwort-Einfügen (Insert) oder Löschen (Delete) gepflegt (Zeitzone Berlin). Ein 2-Tage-Schwellenwert verhindert das Zurücksetzen bei der automatischen 90-Tage-Bereinigung.
- **Synchronität**: `time_sync_5min_count` und `time_sync_1min_count` erfassen zeitnahe Beantwortungen des Paares. `both_long_answers_count` trackt gleichzeitige detaillierte Freitext-Beantwortungen.
- **Kalender & Harmonie**: `answered_valentines`, `answered_new_years` fangen Feiertage ab. `perfect_rankings_count` und `perfect_match_days_count` zählen perfekte inhaltliche Übereinstimmungen.
- **Matches & Tiefe**: `total_matches` wird bei Übereinstimmungen inkrementiert. `long_answers_count` zählt Antworten > 150 Zeichen.
- **Anstupser & Avatare**: `nudges_sent` und `avatar_change_count` werden über BEFORE UPDATE-Trigger erfasst.
- **Journal & Rebuilds**: `journal_views` wird via Client-RPC erhöht. `streaks_rebuilt` wird erhöht, wenn eine fortlaufende Antwortserie nach einem Abbruch wieder auf 7 Tage anwächst.

Die Meilenstein-Prüfung bewertet alle 20 Metriken parallel und schaltet Auszeichnungen freischaltungsdatum-genau frei.

8. Web-Push Notifications & Edge Function Integration

Die PWA-Push-Infrastruktur nutzt Deno-basierte Supabase Edge Functions für kryptografisch gesicherte Benachrichtigungen.

8.1 Der Push-Notification-Dienst (send-push-notification)

Dieser Deno-Dienst empfängt API-Requests vom Client und bereitet das Notification-Payload-JSON (Titel, Nachricht, Ziel-URL) vor. Die eigentliche Übertragung erfolgt über das standardisierte Web-Push-Protokoll directly an den Push-Service des Empfänger-Browsers (z. B. Google FCM, Mozilla autopush).

8.2 Kryptografische Absicherung (VAPID & AES-128-GCM)

Da Deno keine Node.js-Bibliotheken nativ unterstützt, wurde die Web-Push-Verschlüsselung vollständig über die Deno-interne **Web Crypto API** realisiert:

1. **VAPID-Signierung (ES256)**: Ein JSON Web Token (JWT) mit der Ziel-Audienz (Push-Endpoint des Browsers) und einer Ablaufzeit von 12 Stunden wird mit dem privaten VAPID-Schlüssel (ECDSA P-256) signiert. Der öffentliche VAPID-Schlüssel wird als Header-Information mitgeschickt.
2. **Schlüsselaustausch (ECDH)**: Die Edge Function generiert ein temporäres ECDH-Schlüsselpaar. Über das empfangene Client-Schlüsselpaar (p256dh und auth) wird ein gemeinsames Geheimnis (Shared Secret) berechnet.
3. **Schlüsselableitung (HKDF)**: Das System leitet über HKDF-SHA-256 den symmetrischen Content Encryption Key (CEK) und ein Nonce ab.
4. **Verschlüsselung (AES-128-GCM)**: Die Payload wird verschlüsselt und als binärer Datenstrom gesendet. Der Push-Service leitet die Nachricht an das Gerät weiter. Der lokale Service-Worker (sw-push.js) entschlüsselt die Payload und zeigt den System-Toast an.

9. Systemdatenfluss & Datenarchitektur

Nachfolgend wird der typische Ablauf einer Interaktion visuell dargestellt, um das Zusammenspiel zwischen Frontend-Aktionen, Trigger-Prozessen und Hintergrund-Diensten zu verdeutlichen.

Phase	Akteur	Aktion & Systemreaktion
1. Start	Client App.tsx	Prüft Session. Lädt Profildaten. Fragt nach heutigen Fragen in daily_questions. Trigger-Prefetching für morgen.
2. Beantwortung	Client Questions.tsx	Nutzer beantwortet 3-4 Fragen. UI führt optische Verschlüsselungs-Animation aus und speichert Daten via Supabase Client.
3. Persistierung	Postgres DB	Speichert Zeile in public.answers. Löst Trigger 'on_answer_update_streak' aus, welcher Streaks & History aktualisiert.
4. Notification	Edge Function	Questions.tsx ruft 'send-push-notification' auf. VAPID verschlüsselte Push-Nachricht wird an Partner gesendet.
5. Auswertung	Edge Function	Bei Aufruf von StatsModal.tsx berechnet 'calculate-stats' über gte-small Embeddings & Rangabstände die Beziehungsdaten.

9.1 Fazit

Dank des Server-Driven UI-Prinzips, der server-seitigen Trigger für geschäftskritische Zustände (Streaks, Koppelungen) und der Auslagerung komplexer mathematischer Modelle in isolierte, bedarfsgesteuerte Edge Functions ist die Bisou App extrem ausfallsicher, performant und leicht erweiterbar. Alle Kernkomponenten arbeiten autark und greifen über sichere RLS-Policing-Mechanismen ineinander.